

Three ways to search binary embeddings

A guide to the algorithms, their costs, and the evidence

11 September 2026

1 What are we trying to make faster?

A search index receives a query and returns a small set of documents. If we calculate a score for every document, finding the best results is straightforward. The difficulty is doing this quickly when the collection grows. We can skip documents, but then we need to know how much work the shortcut costs and whether it skips the results we wanted.

This report compares three ways to search the same binary document embeddings. Method A opens selected clusters and scans their documents. Method B opens clusters and splits their documents using bitplanes. Method C opens clusters and looks up bit patterns, beginning with the pattern preferred by the query. We will call C *weighted key lookup*; it is the concrete version of the idea of starting at the ideal answer and “walking backwards.”

Our question has three constraints: a document count, a RAM budget, and a required recall. Recall measures how many of the correct reference results the search recovers. We want the lowest latency that satisfies those constraints. The number of calculations matters, but so does the memory each calculation reads. Avoiding a document score can introduce other work.

The experiments give three different outcomes. On the tested real embeddings, neither alternative established a speed advantage over a well-tuned scan. On constructed queries whose important coordinates stayed in one prefix, key lookup had a small advantage over a closely related directly routed scan. When the important coordinates changed with each query, bitplane branching had a large advantage in the constructed case. Understanding those conditions is more useful than declaring one method the winner everywhere.

We begin with one query and six documents, then count the work and storage for each method. The second half explains recall, parameter selection, and the measurements. The final calculation asks what could happen at a billion documents. Those larger latency numbers are forecasts; the complete native indexes were tested through eight million documents.

1.1 One query, six documents, one score

An embedding is a vector of numbers produced from text. We keep the query vector as numbers, but replace each document coordinate with its sign. A stored bit of 1 means +1; a bit of 0 means −1. This is *binary quantization*. For the example, let

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

The score is the dot product: multiply corresponding coordinates, then add. The query would prefer a document with signs 11001, because every contribution would be positive. That document scores $0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5$.

Document ID	Stored signs	First-bit cluster	Full score
0	01001	0	1.1
1	00100	0	-1.3
2	10100	1	0.1
3	11001	1	2.5
4	11000	1	1.9
5	00011	0	-1.1

The correct top two are documents 3 and 4. We use smaller document IDs first if scores tie. Every method will be checked against that same rule. The script `examples/small_search.py` runs this example against the native implementations.

Why should query magnitude guide a branch? At coordinate 1, matching the query contributes $+0.7$ and mismatching contributes -0.7 . The mismatch lowers the score by 1.4. At coordinate 5, the corresponding loss is only 0.6. Large magnitudes therefore make expensive mistakes. For a query with d coordinates, write the total ideal score as $Q = \sum_{i=1}^d |q_i|$. Writing b_i for the decoded sign ($+1$ or -1), if the mismatching coordinates have total magnitude p , then

$$S(q, b) = Q - 2p, \quad p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|. \quad (1)$$

For document 4, only coordinate 5 mismatches: $S = 2.5 - 2(0.3) = 1.9$. We can rank by the largest score or the smallest mismatch penalty. They give the same order. Ordinary Hamming distance counts mismatches equally; our score weights them by the query magnitudes.

1.2 From text to an index, then back to results

The collection contains N documents. Before queries arrive, we embed the text, choose the representation width d , pack the document signs, and build the index. Each document receives one cluster assignment. A cluster is simply a stored subset of the collection. The index may learn clusters from vectors, or assign documents from a fixed sign prefix.

Matryoshka training makes selected prefixes useful at smaller dimensions [2]. It does not sort the coordinates of each query by magnitude, and it does not guarantee that 24 sign bits preserve its nearest neighbors. Truncating an embedding and taking its signs are separate changes to the representation. Our primary comparison fixes the representation across methods.

At query time, we embed and normalize the query, select clusters, and search their documents. Opening a cluster is one *probe*. With C clusters and P probes, balanced clusters would contain about NP/C selected documents. Real clusters need not be balanced, so the actual count is the sum of the selected cluster sizes. Setting $C = P = 1$ searches the whole collection.

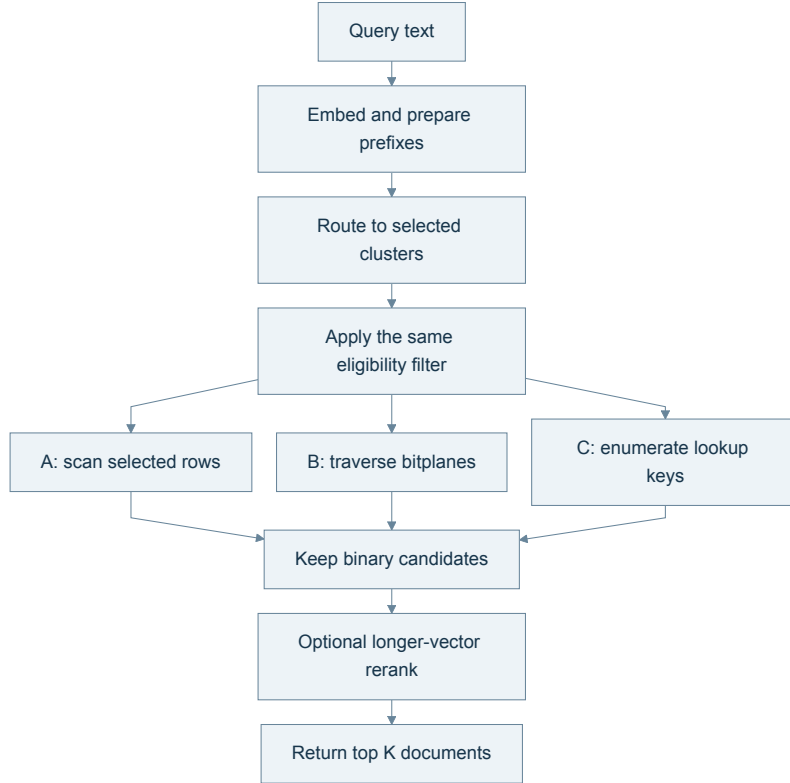


Figure 1: The common query flow. The three local search methods share the same query and selected clusters when we isolate their work. Longer-vector reranking is optional; it is excluded from the primary binary-score comparison.

A production pipeline can apply document filters and rerank candidates with longer vectors or another model. These stages belong in a complete request measurement when enabled. In this study, the primary index comparisons use cached query embeddings, the same binary score, and no float reranker. Later we measure actual text queries as complete requests.

Exa’s published vector-database description motivates the cluster-and-scan design and compact scoring [1]. Our A is a local baseline with that broad structure. It is not a reproduction or benchmark of Exa’s production system. In particular, sign-prefix routing and learned IVF clusters are different choices; a k -bit prefix defines up to 2^k patterns, not k clusters.

2 Three ways to search the selected documents

2.1 A: cluster, then full scan

We first use a scan because its behavior is easy to check. In the six-document example, coordinate 1 defines two clusters. We open cluster 1 and then cluster 0. A reads the codes of documents 2, 3, 4, then 0, 1, 5, computes their full scores, and keeps the best two. The order of visits does not change the final ranking.

A *top-K heap* stores the best K results found so far, with the worst retained result available

for replacement. For $K = 2$, after documents 2 and 3 it holds scores 0.1 and 2.5. Document 4, scoring 1.9, replaces document 2. None of the remaining documents improves the pair.

Listing 1: Full scan. Every selected document receives a full score.

```
scan(query, selected_clusters, K):
    table = prepare_score_table(query)
    best = empty_top_k(K)
    for cluster in selected_clusters:
        for document in cluster.documents:
            score = full_score(table, document.code)
            best.offer(score, document.id)
    return best.sorted_by_score_then_id()
```

The score table avoids multiplying every sign separately. For each group of four coordinates, we precompute the contribution of all 16 possible sign patterns. A 256-bit document then needs 64 table terms. A lookup still involves reading bits, extracting a pattern, loading a value, and adding it to the score. A table term is not a single CPU instruction.

More generally, for g coordinates per score-table group, there are $G = \lceil d/g \rceil$ groups and $G2^g$ stored table values, allowing padding in the final group. Preparing the table is query work, shared by all three methods. Changing g trades table size against the number of terms per document. The measured implementation uses $g = 4$.

Let $\mathcal{P}$ be the selected clusters and n_c the size of cluster c . A scores

$$F_A = \sum_{c \in \mathcal{P}} n_c$$

documents and evaluates GF_A table terms. A useful local time model is

$$T_A^{\text{local}} \approx T_{\text{table}} + F_A c_{\text{scan}} + T_{\text{select}}(F_A, K). \quad (2)$$

Here c_{scan} is the cost of reading and scoring one contiguous document at the chosen width. Selection is written separately to show its role. If a measured per-row coefficient already includes selection, we must not add it again. A worst-case heap bound is $O(F_A \log K)$, but not every score changes the heap.

2.2 The scan's storage and temporary memory

Let w be the number of bits in a packed word and $u = w/8$ its bytes. A code occupies $u \lceil d/w \rceil$ bytes; let b_{id} be the document-ID width. The row payload is

$$M_{\text{rows}} = N(u \lceil d/w \rceil + b_{\text{id}}). \quad (3)$$

For $d = 256$, $w = 64$, and eight-byte IDs, each row needs $32 + 8 = 40$ bytes. At one million rows, that is 40 MB in decimal units. Cluster directories, vector spare capacity, and allocator bookkeeping require additional memory.

Routing has its own storage. For example, C float centroids of width d_r need $4Cd_r$ bytes for their coordinates. Exact centroid comparison examines Cd_r coordinates per query,

plus the work of choosing the best P centroids. A sign-prefix router has a different cost. Routing cannot be assigned one universal price independent of its implementation.

The query also needs temporary storage: its vector, the score table, selected cluster IDs, and the current results. Denote this shared allocation by Q_0 . Its main terms scale as $O(d + G2^g + P + K)$. Include Python/native result arrays if they coexist. The serving memory for A is stored rows, routing and container allocation, plus Q_0 . A build can peak higher because the input arrays and newly arranged index exist together. File size, logical payload, and process RAM therefore answer different questions.

The scan has a useful hardware property: codes in a cluster are read consecutively. The CPU can fetch future cache lines while it works on the current ones. SIMD instructions can process several values together. These effects change the measured price of a row, even though the number of scored documents is unchanged. They explain why reducing arithmetic alone is not enough to demonstrate a faster search.

2.3 B: arrange the same bits by coordinate

To avoid some full scores, B needs a way to select documents by a query bit. It stores a *bitplane*: one coordinate across all documents in a cluster. A bitplane is the same sign data turned into columns. B keeps the packed document rows as well, so it can score a selected candidate with the common full scorer.

In cluster 1 of our example, the local document order is [2, 3, 4]. Their codes are 10100, 11001, and 11000. The plane for coordinate 2 is 011: documents 3 and 4 have that bit set. The plane for coordinate 3 is 100. The query prefers coordinate 2 to be 1 and coordinate 3 to be 0; both selections keep documents 3 and 4.

A candidate mask records which local document positions remain eligible. With parent mask A and plane B_i , a positive-query split makes

$$A_{\text{match}} = A \& B_i, \quad A_{\text{other}} = A \& \neg B_i.$$

For a negative query coordinate, the matching and other children swap. Each result is intersected with the parent, so padding positions never become real documents.

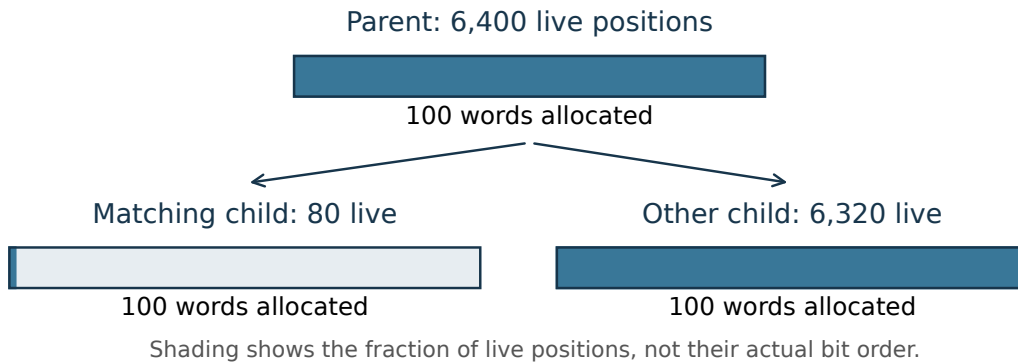


Figure 2: Membership shrinks, but a dense mask keeps its physical width. Each child still has 6,400 positions, or 100 words. This larger illustration counts storage; the six-document example explains the returned IDs.

2.4 Follow the promising branch, keep the alternative

The query's magnitudes order coordinates as 1, 4, 2, 3, 5. Within cluster 1, coordinate 1 is already fixed and all three documents agree with the query on coordinate 4. Splitting on coordinate 2 then separates [3, 4] from [2]. The first child has no new mismatch penalty; the second adds 0.5. We visit the more promising child first.

Visiting it first does not justify deleting the alternative. A document may match this bit but disagree on several later bits. B stores unexplored children in a queue, ordered by a lower bound on their mismatch penalty. From Equation 1, a node with known penalty p has score upper bound $Q - 2p$: unexamined coordinates are allowed their best signs. If this upper bound is strictly below the worst retained top- K score, the node cannot help. Equal-score possibilities still need the ID tie rule; the native check is conservative around floating-point comparisons.

Listing 2: Bitplane search. The queue preserves alternatives to the preferred path.

```
branch(query, clusters, K, leaf_size, node_budget):
    table = prepare_score_table(query)
    order = coordinates_by_decreasing_abs(query)
    queue = one full candidate mask per selected cluster
    best = empty_top_k(K)
    while queue is not empty:
        node = remove_lowest_penalty(queue)
        if cannot_improve(node, query, best): continue
        if node_budget_is_reached(): return best
        if node.count <= leaf_size or no_bits_remain(node):
            score_each_set_document(table, node.mask, best)
        else:
            matching, other = split_on_next_bit(node, order)
            queue_nonempty_children(queue, matching, other)
    return best.sorted_by_score_then_id()
```

This listing shows the search policy; the native source also handles counters, ownership, and numerical tolerance. A leaf is a node small enough to score directly. The leaf size L therefore decides when to stop splitting. A node budget limits the work but may stop before the bounds establish the answer. With no work limit and safe bounds, the method can finish exactly within the selected clusters. Unprobed clusters are still outside that guarantee.

A probability parameter can occasionally choose another queued node instead of the best bound. This changes the order of exploration, not the score. Under a fixed budget it may help or hurt recall, so we test it with seeds rather than assume an improvement. The tested real-data settings did not show a consistent gain from adding random exploration.

2.5 Count the full-width work

For cluster c , each dense mask spans $W_c = \lceil n_c/w \rceil$ words. Let J_c count its splits and E_c the leaf masks whose set bits are enumerated. The full-width visits are

$$V_{\text{split}} = \sum_{c \in \mathcal{P}} J_c W_c, \quad V_{\text{leaf}} = \sum_{c \in \mathcal{P}} E_c W_c. \quad (4)$$

A leaf loop still checks zero words to find out that they are empty. The number of set bits then determines how many documents are scored. Omitting leaf-word visits was one source of an overly optimistic early cost estimate.

Suppose a 6,400-document cluster has been narrowed to 80 candidates. Its mask still contains 100 words. Ten further dense splits visit 1,000 words, even if later masks contain only a few set bits. Each visit reads the parent and plane, builds children, and writes results. Allocating masks and maintaining the queue cost time as well.

If F_B documents are eventually scored, a practical decomposition is

$$\begin{aligned} T_B^{\text{local}} \approx & T_{\text{table}} + T_{\text{order}} + V_{\text{split}} c_{\text{split}} + V_{\text{leaf}} c_{\text{leaf}} \\ & + F_B c_{\text{gather}} + T_{\text{queue}} + T_{\text{select}}(F_B, K). \end{aligned} \quad (5)$$

The per-word split price should include the chosen mask allocation policy. Scattered document reads may have a different per-row price from contiguous scan reads. If a measured leaf timer already includes finding set bits, fetching rows, and scoring them, use that combined term instead of adding these stages a second time.

B's additional stored plane payload is

$$M_{\text{planes}} = du \sum_{c=1}^C \lceil n_c/w \rceil. \quad (6)$$

For one large cluster this is approximately $Nd/8$ bytes. Many small clusters add padding. During the query, if $A_c(t)$ masks for cluster c coexist at time t , the mask payload peak is $u \max_t \sum_c A_c(t) W_c$. Add queue-node allocation and shared scratch. Counting just the currently visited mask misses the alternatives waiting in the queue.

2.6 C: begin with the query's key

B chooses a sequence of columns from the query. C instead builds a directory over a fixed set of h coordinates. Within each cluster, documents with the same short key are stored together. A directory entry maps an occupied key to a start position and count. The existing document IDs identify that posting range; the implementation does not add a second ID array for postings.

Use coordinates 2 and 3 as the key in the running example. The query prefers 10. Flipping coordinate 3 costs 0.4, flipping coordinate 2 costs 0.5, and flipping both costs 0.9. Thus the weighted order is 10, 11, 00, 01.

Query key: coordinates 2 and 3 = 10

Key order	Known penalty	Cluster 0 IDs	Cluster 1 IDs
10	0.0	0	3, 4
11	0.4	empty	empty
00	0.5	5	empty
01	0.9	1	2

Figure 3: One shared key sequence is looked up in both selected clusters. The ideal key already finds the top two, but that fact alone is not a safe stopping rule. Empty lookups still cost work.

The ideal key returns document 0 from cluster 0 and documents 3 and 4 from cluster 1. Their full scores are 1.1, 2.5, and 1.9, so the best pair is [3, 4]. The next key’s known penalty is 0.4, giving a full-score upper bound of $2.5 - 2(0.4) = 1.7$. That is below 1.9, so in this example a safe bound can stop before the empty 11 lookups. Figure 3 shows the wider sequence to make the directory and empty keys visible; an unlimited walk without that bound would inspect them too.

Why not always stop after finding K documents? A short key ignores the other coordinates. A document matching the key may have a poor full score, while another key may contain a better document. C therefore scores the full code and keeps an upper bound for unvisited keys.

Listing 3: Weighted key lookup. Generate the key sequence once; charge every cluster lookup.

```

keys(query, clusters, K, candidate_target, lookup_limit):
    table = prepare_score_table(query)
    ideal = signs_of_fixed_key_coordinates(query)
    queue = [empty_flip_set_with_zero_penalty]
    best = empty_top_k(K)
    while queue is not empty:
        if remaining_bound_cannot_improve(queue, query, best): break
        state = remove_lowest_penalty(queue)
        key = ideal XOR state.flip_mask
        for cluster in clusters:
            if lookup_limit_is_reached(): return best
            count_one_lookup_including_misses()
            for document in cluster.directory.get(key, []):
                best.offer(full_score(table, document.code), document.id)
        if candidate_target_is_reached(): return best
        add_next_unique_flip_sets(queue, state)
    return best.sorted_by_score_then_id()

```

The queue enumerates combinations in increasing total mismatch weight. It is not restricted to increasing numbers of flipped bits: two low-weight flips can cost less than one high-weight flip. The native implementation uses a subset-generation rule that avoids generating the

same combination repeatedly. For every unvisited key, $Q - 2p_{\text{key}}$ is a safe full-score upper bound, because omitted coordinates may all match. That bound can be loose.

A positive candidate target or lookup limit can stop early and lose recall; zero disables a limit in the native API. The target is checked after completing a key across selected clusters. The lookup limit may stop during that cluster loop. Exact local completion requires exhausting potentially improving keys or proving that none remain, while respecting score ties.

2.7 What the key directory costs

Let H count actual directory lookups and F_C the full scores. With E generated keys and no limit cutting a cluster loop, $H = PE$. An empty key contributes to H but not F_C . A time model for the implemented policy is

$$T_C^{\text{local}} \approx T_{\text{table}} + T_{\text{key order}} + T_{\text{key queue}} + Hc_{\text{lookup}} + F_Cc_{\text{posting}} + T_{\text{select}}(F_C, K). \quad (7)$$

Posting rows are contiguous within a key, but many short ranges still differ from one long scan. Queue costs depend on the live generated states. We must not charge P copies of the queue: this implementation shares one stream across clusters.

For U_c occupied keys in cluster c , the logical directory payload is $\sum_c U_c(b_k + 2b_o)$: a key, start, and count. The current key field uses four bytes and each range field uses eight. Allocated memory is larger. The C++ directory uses separate chaining, with one node per occupied entry and a bucket-pointer array. If B_c is the number of hash buckets, its allocation includes $\sum_c (U_c b_{\text{node}} + B_c b_{\text{ptr}})$, plus container and allocator overhead. The 20-byte field sum is not an allocated-node measurement.

A wider key can shorten posting ranges while making useful keys harder to reach. A very short key finds documents easily but may leave nearly all the scoring work. Finding a nonempty key and finding the best documents are different events. Looking up a relaxed pattern such as $1*0$ also needs an explicitly different operation: a hash table over exact keys does not answer a wildcard request in one lookup.

3 Recall and the choice of parameters

3.1 Recovering the reference results

A fast query is useful only if it returns the results we intended to find. Let $G_K(q)$ be the reference top- K document IDs from a complete scan, and $A_K(q)$ the IDs returned by a method. For $K \leq N$, its recall for query q is

$$R_K(q) = \frac{|G_K(q) \cap A_K(q)|}{K}. \quad (8)$$

Our six-document example has $G_2 = [3, 4]$. Returning $[3, 0]$ recovers one of those two IDs, so recall is 50%. Returning $[3, 4]$ gives 100%. Returning many candidates is not itself high

recall; the candidate set must contain the reference IDs, and the final selection must keep them.

This definition needs a specified reference. Our main experiments use the full binary-document score in Equation 1. A float-vector reference would measure the additional loss from quantization. Human relevance labels answer a different question: whether the text is useful for a query. High binary-neighbor recall does not prove high relevance or accurate answers from a language model.

There are two places to lose a reference document: routing can exclude its cluster, and local search can miss it inside an opened cluster. To express this precisely, fix a query and choose one of its K reference IDs uniformly. Let V mean that its cluster is visited and L mean that the local method returns it. Then

$$R_K(q) = \Pr(V \mid q) \Pr(L \mid V, q). \quad (9)$$

This is a conditional probability identity, not an independence assumption. Averaging it over queries does not generally equal the product of the two separately averaged probabilities. A's exact local scan has conditional survival one; it can still lose results through routing. B and C have the same routing limit, plus any loss caused by local work limits.

3.2 Derive one useful distributional model

We can calculate expected recall if we specify how documents and query weights are distributed. Start with independent document signs, each equally likely to be 0 or 1. Choose m strong query coordinates of magnitude a ; give the remaining $d - m$ coordinates magnitude ϵ . Require

$$a > (d - m)\epsilon. \quad (10)$$

One strong mismatch then costs more than all possible weak mismatches together. Consequently, every document matching all strong coordinates outranks every document with a strong mismatch. This is a deliberately constructed condition. Matryoshka training alone does not imply it.

A random document matches those m signs with probability $p = 2^{-m}$. Let Z count the matching documents among N independent rows. It has a binomial distribution:

$$\Pr(Z = z) = \binom{N}{z} p^z (1 - p)^{N-z}, \quad \mathbb{E}[Z] = Np.$$

Suppose we open only this ideal strong-sign group and score every row inside it. If $Z \geq K$, all top- K documents are present. If $Z < K$, it supplies Z of them. The query recall is $\min(Z, K)/K$, so

$$\mathbb{E}[R_K] = \frac{1}{K} \sum_{z=0}^N \min(z, K) \binom{N}{z} p^z (1 - p)^{N-z} = \frac{1}{K} \sum_{i=1}^K \Pr(Z \geq i). \quad (11)$$

The second form adds the probabilities of recovering at least one result, at least two, and so on through K . It avoids summing over every possible corpus count.

A small calculation shows why the distribution matters. With $N = 2$, $m = 1$, and $K = 1$, the ideal group is empty with probability $1/4$, contains one document with probability $1/2$, and contains two with probability $1/4$. Expected recall is $0(1/4) + 1(1/2) + 1(1/4) = 3/4$. Although the average group size is one, treating that average as a guaranteed count would incorrectly predict 100% recall.

Ideal-group size Z	Probability	Recall@1
0 matches	1/4	0
1 match	1/2	1
2 matches	1/4	1

Figure 4: Two independent documents and one required matching bit. Opening only the matching group succeeds with probability $3/4$. The empty-group outcome explains why the mean group size is insufficient.

At $N = 10^6$ and $m = 13$, the average ideal group contains about 122 rows. With $K = 100$, some groups may still be too small. A complete B or C search can continue into other groups; Equation 11 describes stopping after the ideal group, not every possible search policy. Increasing a work budget changes which additional groups are inspected and requires accounting for their scores and sizes.

For general queries, a prefix’s fraction of dimensions is not its fraction of useful ranking information. Correlated document bits, different query magnitudes, and approximate routing change candidate survival. A normal approximation to prefix and full scores may be useful, but it needs calibration and an independent check. In the earlier local Nomic check, its recall errors reached about 9.6 percentage points and failed the proposed two-point criterion. We therefore do not use that approximation to claim real-data recall at a billion rows.

3.3 Which parameters can we change?

Some settings change the stored index: code width d , cluster count C , routing method, and C’s key coordinates and width h . Others change a query: probes P , B’s leaf size and node budget, C’s lookup and candidate limits, and the requested K . Changing d also changes the reference ranking, so all methods must be compared again at that same representation.

We do not need to sweep every possible combination blindly. A cost equation can identify a useful region. For B, suppose each split halves the candidate count, one promising path is followed, and the masks retain width $W = \lceil N/w \rceil$. After j splits, expected scoring work is $N2^{-j}$ rows. Ignoring queue changes for the moment, write

$$T_B(j) \approx T_0 + jWc_{\text{split}} + Wc_{\text{leaf}} + N2^{-j}c_{\text{score}}. \quad (12)$$

One more split saves about $N2^{-(j+1)}$ scores but adds a full-width split. It is worthwhile in this model when the saved scoring time exceeds Wc_{split} and the additional queue cost.

For example, at $N = 6,400$ and $j = 5$, another balanced cut saves about 100 scores. We compare the time of those 100 scores with a 100-word split; we do not equate the counts. Treating j as continuous gives a starting point:

$$\frac{dT_B}{dj} = W c_{\text{split}} - N \ln(2) 2^{-j} c_{\text{score}} = 0, \quad j^* = \log_2 \left(\frac{N \ln(2) c_{\text{score}}}{W c_{\text{split}}} \right).$$

We then check valid neighboring integer depths, the score assumptions, and the required recall. The formula is conditional on a single useful balanced path; it does not cover arbitrary branch exploration. Since W grows with N , more documents do not automatically imply a larger optimal depth. The dense-mask work still grows with the collection.

The strong-coordinate model suggests a leaf threshold too. At one million rows, a 13-bit ideal group has mean about 122 and standard deviation about 11. A threshold of 128 sometimes forces an extra weak-coordinate split. The rough margin $122 + 3(11) \approx 155$ suggested testing $L = 160$. The preceding 12-bit group averages about 244 rows, so it usually still splits. The experiment checked this setting rather than treating the approximation as a guarantee.

The complete choice for method a is

$$\min_{\theta \in \Theta_a} T_a(\theta) \quad \text{subject to} \quad R_a(\theta) \geq r, \quad M_a(\theta) \leq M_{\text{limit}}. \quad (13)$$

The vector θ contains that method’s settings. A derivative can help where the functions are known, but cluster occupancy and routing recall are not determined by N and RAM alone. In practice, we measure promising settings, retain the feasible ones, and compare the best *tested* choices. We must give the scan enough routing choices and let B stop splitting when a scan-like leaf is cheaper.

4 What the experiments establish

The work formulas tell us what each method must do. Measurements answer the next question: does the scoring it avoids pay for the search it adds? We first compare real embeddings, then use constructed inputs to isolate the importance of the query’s strong coordinates. The saved source record identifies the runs behind these comparisons [5].

4.1 Make the comparison fair

A, B and C receive the same documents and floating-point queries. They use the same native weighted binary scorer, return the top 100, and break score ties by document ID. Recall is recovery of that exact binary ranking. There is no float reranker or human relevance judgment in these comparisons.

The machine is an Apple M5 Max with 128 GiB of physical memory. Each worker uses one CPU thread and runs one query at a time, under a separate 32 GB decimal memory budget. A fresh process builds a configuration, warms it, and measures routing plus search. This outer timer includes the call from Python into C++. Cached-vector measurements

exclude text encoding. Repeated process blocks help expose timing variation; the recorded power settings do not prove that CPU frequency remained fixed.

We tune parameters on one set of query IDs, freeze the choices, and evaluate them on different IDs. Each method may choose its own fastest tested setting that meets the recall requirement. A gets intermediate probe counts and direct routing by sign keys. B gets large leaves, so it can score a selected cluster without making unnecessary splits. Without these options, a poor baseline or a forced traversal could decide the result before the timing begins.

The real study retains two selection rules. One accepts settings whose average tuning recall reaches the target. The conservative rule requires a lower estimate from repeated resampling of tuning queries to reach it. This leaves more margin, but cannot promise recall on future queries. A supported speedup also requires both methods to meet the target and the paired 95% interval for their time ratio to lie above one. Queries and whole process blocks are resampled together; timing repeats do not count as additional independent queries.

4.2 Real embeddings did not establish an alternative speedup

The real study uses 256-bit document codes from Nomic embeddings of MS MARCO passages [4, 3]. After combining 6,821 tuning settings, the frozen choices were evaluated on 200 query IDs excluded from the earlier 300-query collection. All 36 conservative target points met their requested recall on this sample; only 7 of 36 points chosen by average tuning recall did.

The table shows the conservative results at one million documents. Read each row as a common minimum recall requirement, followed by each method’s median routing-plus-search time. The achieved recall happens to be equal across methods in these rows.

Target	A (ms)	B (ms)	C (ms)	Recall (%)
80%	0.675	0.699	0.751	82.53
90%	1.634	1.686	1.880	91.73
95%	3.360	3.430	3.706	95.47
99%	9.580	9.965	11.013	99.25

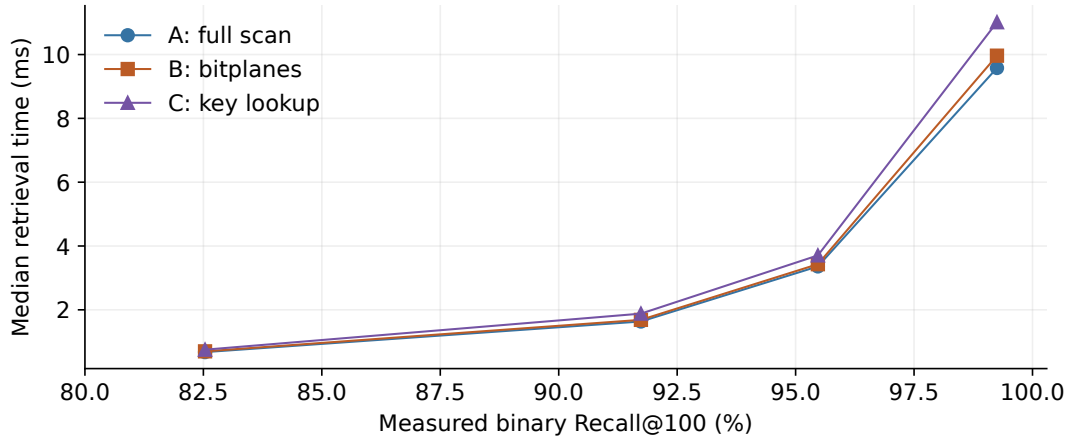


Figure 5: Four frozen recall-target choices on one million real passage embeddings and 200 independent query IDs. Lines connect the tested settings for readability; they do not predict intermediate settings. At each shown recall, A has the smallest median.

None of the 48 evaluated comparisons across the real study established a B or C speedup under the stated rule. The conclusion applies to this data, score, implementation and tested parameter range. Fewer fully scored documents did not produce a supported reduction in the complete retrieval time.

A later control gives C a one-bit key, A’s routing layout, and unlimited candidate and lookup budgets. A and B keep their frozen settings. It uses 100 further query IDs, excluding all 500 previously used IDs. A, B and C measured 9.541, 9.855 and 10.545 ms, respectively. All three achieved 98.95% recall, below the requested 99%. We retain that miss without rounding it into a pass. It is a separate sample from the earlier successful 200-query evaluation, and no parameter was changed in response to its outcome.

4.3 Constructed data shows when skipping scores can help

The controlled inputs make a specific distinction testable. Document signs are independent and balanced across 256 coordinates. Each query has 13 strong coordinates of magnitude one; the remaining coordinates have magnitude 0.0001. Matching every strong sign therefore beats any document that mismatches one, regardless of the weak signs. In the fixed case, the strong positions stay the same. In the changing case, each query chooses different strong positions uniformly without replacement. These are constructed vectors, not measured text embeddings.

The final follow-up uses 40 tuning queries and 20 separate evaluation queries, with three process blocks and three timing repeats. It adds neighboring key widths, cluster counts and leaf sizes, then remeasures earlier choices. The results below use one million documents and the 99% requirement. All methods reach 100% recall except A in the changing case, at 99.95%.

Strong positions	A (ms)	B (ms)	C (ms)
Fixed	0.0140	0.0160	0.0128
Changing	24.9235	0.1446	14.6301

With fixed positions, A selects two of 16,384 clusters by direct routing; C uses one physical cluster and a 13-bit key. C’s paired speedup estimate is 1.098, with a 95% interval of 1.066–1.147. The time difference is about 1.25 microseconds. Both methods select documents through the same strong signs. C combines key enumeration and scoring in one native call; A routes and scans through separate calls. The small measured advantage concerns this implementation of closely related candidate selection.

Changing positions prevent a fixed key from consistently selecting the strong signs. B can follow them because it orders coordinates using each query’s magnitudes. Its selected setting uses one cluster, unlimited traversal and a leaf size of 160. A opens 255 of 256 centroid clusters. B’s paired speedup estimate is 172.3, with a 95% interval of 169.1–177.2. C uses a 14-bit key, but its paired interval against A includes one; its lower median alone does not establish an advantage.

The leaf choice follows the occupancy calculation. At one million rows, the ideal 13-bit group has mean size about 122 and standard deviation about 11. A threshold of 128 can force another split on a weak coordinate. A threshold of 160 leaves more room to score that group. The experiment checks this distribution-based suggestion. It does not establish that 160 is best for arbitrary embeddings, sizes or machines.

4.4 Check the answer, the work, then the predicted time

An independent answer comes first. The checks calculate scores directly from document signs and sort by score and ID. Larger runs count strong and weak mismatches from packed bits, avoiding a huge expanded matrix. Every complete native call in the scaling check returned the independent reference ranking.

Next, predicted work is compared with native counters. Across the earlier controlled tuning and evaluation runs, 447 applicable per-query predictions matched exactly. These checks include full-width split and leaf visits, scored rows, key attempts including empty lookups, and tracked mask peaks where applicable. Queries outside the formulas’ stated conditions remain in the measurements; they are not counted as successful predictions.

Correct counts do not give every row the same time cost. In a 16-million-document buffer, scoring about 1,958 scattered rows took 0.159 ms with reused candidates and 0.455 ms with changing candidates. Candidate generation was outside the timer, but changed which memory had been accessed recently. Fits trained on one- and four-million-row buffers underestimated a 16-million-row check by as much as 45.2% for reused candidates and 27.6% for changing candidates. Both failed fits remain saved. The largest row store was about 640 MB, so these checks do not validate access costs for a 40 GB store.

The later complete-call model uses one latency median per corpus size. This corrects an earlier fit that gave fast individual queries too much weight when predicting a median. For C’s small postings, the revised model also accounts for result-selection work. B’s leaf threshold grows as $L = 160N/10^6$, keeping the intended strong-coordinate depth instead of forcing more cuts as the corpus grows.

The revised four-million-row check had already been inspected, so it is diagnostic. The coefficients were frozen before the separate eight-million-row run below. Positive error means the prediction was slower than the measurement.

Positions	Method	Measured (ms)	Predicted (ms)	Error
Fixed	A	197.029	203.016	+3.0%
Fixed	B	0.854	0.884	+3.6%
Fixed	C	0.0481	0.0518	+7.7%
Changing	A	196.664	197.816	+0.6%
Changing	B	0.900	0.859	-4.5%
Changing	C	202.767	207.086	+2.1%

All six checks met the earlier 20% criterion. They test transfer to a larger corpus under the same prescribed workload and existing query IDs. These are model-check configurations, not the tuned comparisons in the preceding table. The largest observed error, 7.7%, is not a guaranteed bound for future queries or a billion documents.

4.5 Measure actual text requests

A user supplies text, so retrieval time covers only part of the wait. The final check runs 20 existing independent-query texts through the cached local Nomic model on MPS, normalizes each vector, routes it and calls native search. Batch size is one. Three blocks and two repeats give 120 requests per method, or 360 requests in total. Each sequence shape is warmed; model loading, index building and reference creation are outside the request timer.

The table reports stage medians and the separately measured request total. All three methods use the earlier frozen settings and achieve 99.45% binary recall on this sample.

	Encode (ms)	Retrieve (ms)	Total (ms)	Total p95 (ms)
A	6.488	9.406	16.313	19.201
B	6.500	9.651	16.303	19.217
C	7.543	11.201	18.912	21.898

The median is the middle time; p95 is the 95th-percentile time. The tiny A/B total difference does not establish a speedup. Adding stage medians need not reproduce the total median: the request with the middle encoding time need not have the middle retrieval time. Each request’s total must be measured before taking its median.

These timings apply to real Nomic text queries. The constructed strong-coordinate vectors do not come from this encoder. Adding its time to their search measurements would not create a measured text-to-results pipeline.

5 Memory limits and larger collections

5.1 Put the stages back together

We can now describe one complete request. Let T_{shared} include query embedding, normalization, routing, and any common filtering. Let T_a^{local} be the local cost for A, B, or C

derived earlier, including its score table. Then

$$T_a^{\text{request}} = T_{\text{shared}} + T_a^{\text{local}} + T_{\text{final}}, \quad (14)$$

where T_{final} contains output work and any enabled reranking. This equation sums stages for the same request. It does not justify adding unrelated stage medians or adding the real encoder’s measured time to constructed vectors that the encoder never produced.

The corresponding memory summary separates stored allocation from live query scratch. Write M_0 for allocated document rows, routing state, and common containers. Let D_C be C’s allocated directory, Q_B B’s extra masks and queue, and Q_C C’s extra key queue. The common query scratch remains Q_0 .

Method	Stored index allocation	Query scratch
A	M_0	Q_0
B	$M_0 + M_{\text{planes,allocated}}$	$Q_0 + Q_B$
C	$M_0 + D_C$	$Q_0 + Q_C$

Actual common allocation can differ with container capacities. Add the runtime and other resident data once when applying a process RAM limit. Do not add overlapping process and GPU accounting blindly. A build-memory limit must also include temporary input and output copies.

5.2 Start with the memory calculation at one billion rows

At $N = 10^9$, $d = 256$, and eight-byte IDs, packed codes occupy 32 GB and IDs occupy 8 GB. These two arrays alone exceed a 32 GB budget. B keeps a second sign representation, adding at least 32 GB before per-cluster padding. Changing the number of probes cannot remove this stored payload.

Layout at 1B rows	Payload floor	Planned query RAM	64 GB budget
A, direct prefix route	40 GB	about 49 GB	Modeled fit
C, global 13-bit key	40 GB	about 49 GB	Modeled fit
B, global cluster	72 GB	about 83 GB	Rejected
B, prefix route, large leaf	72 GB	about 81 GB	Rejected

The planning values include capacity and container allowances and a configurable 1 GB runtime reserve; the reserve is not measured runtime memory. All three fail 32 GB in their current in-memory layouts. All listed planning cases fit 1 TB, but a machine with 1 TB need not have this machine’s memory performance. Compressed IDs or another storage architecture would require a new calculation, not a different label on the same index.

5.3 When a fixed prefix carries the useful information

Under the earlier independent strong-coordinate model, thirteen fixed strong coordinates give an expected ideal group of $10^9/2^{13} \approx 122,070$ rows. This is far more than the requested

100. The distributional calculation controls the small probability of an insufficient group; it is not a claim about ordinary text embeddings.

A can route directly to this group and scan it. C can look up its key and score the same rows. Both cost roughly 3 ms in the calibrated conditional model. B can also use this route and a large leaf, producing a similar time with more stored data. We should not force B through a global billion-row bitmap just to make C appear better. For this case, A and C are closely related ways to obtain the same candidate set, and the model cannot resolve a few-percent gap.

5.4 When the important coordinates change with each query

A fixed prefix may now contain little useful signal. B can choose its split coordinates from the query. Keeping thirteen useful cuts and scaling the leaf size to 160,000 preserves the constructed path. Its last-split mask payload is approximately

$$(13 + 2) \lceil 10^9 / 64 \rceil \times 8 = 1.875 \text{ GB.}$$

The factor 15 counts the simultaneously live path and alternative masks for this prescribed path, including parent/child overlap. Other search histories can need different scratch.

The complete native size model predicts about 0.10 s for this global B path and about 25 s for a global scan. Doubling B’s variable cost gives about 0.20 s; this is a sensitivity example, not a confidence interval. No full billion-document query was measured.

A useful comparison gives A the benefit of routing. If it scores fraction f of the collection, its approximate cost is $T_{\text{route}} + fNc_{\text{scan}}$. Even ignoring routing time, A must score less than about 0.41% of the corpus to undercut the 0.10 s B prediction. It must also meet the recall target at that fraction. If routing produces a small, accurate candidate set cheaply, scanning can win. If it cannot, the query-dependent B path is worth testing.

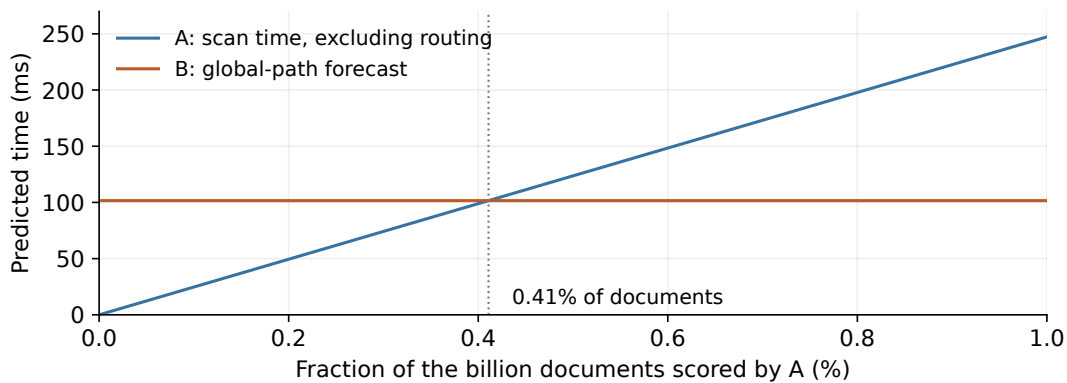


Figure 6: A conditional comparison with sufficient RAM. B is one prescribed operating point. A gets free routing in this line but must still attain the required recall at its scanned fraction. These are forecasts, not measured billion-document timings.

For C, a short key must actually capture important coordinates. In the constructed changing case, a fixed 13-coordinate key misses all thirteen strong coordinates with probability about 49.94%. A nonempty directory lookup can therefore still leave almost

the entire scoring task. Key occupancy alone cannot establish the latency or recall advantage.

5.5 What we can conclude

On the measured real embeddings, the scan remains the supported practical baseline. Under a strong fixed-prefix condition, direct routing and key lookup both avoid most scores. Under changing strong coordinates, B's query-dependent splits produced a large measured advantage in constructed data. Each conclusion has a stated workload and a tested parameter space.

The formulas help explain those outcomes and suggest settings. They do not prove that every possible IVF layout loses, that Matryoshka creates the constructed distribution, or that a local timing coefficient remains exact at a billion rows. The next useful experiment would check whether a real representation has the score concentration and routing behavior required by the favorable case, while preserving the same recall and RAM constraints.

6 From the explanation to the code

The companion repository contains the complete implementations, saved measurements, and the longer derivations. Paths below are relative to its root. The 72-page report remains in `reports/search-math/`; this edition is in `reports/search-math-concise/`.

Question	Start here
Follow the six-document example	<code>examples/small_search.py</code>
Inspect scan and branching	<code>cpp/index.cpp</code> , <code>cpp/score.hpp</code>
Inspect key lookup	<code>cpp/key_index.cpp</code>
Check storage and work formulas	<code>experiments/models.py</code> , <code>experiments/check_controlled.py</code>
Read the final timing checks	<code>experiments/check_native_model.py</code> , <code>reports/search-math/evidence/final-study.json</code>
Calculate the larger cases	<code>experiments/project_costs.py</code> , <code>results/projected-cases-2026-09-11/</code>
Reproduce the study	<code>experiments/README.md</code> , <code>results/README.md</code>

After installing the project as described in its root README, we can run the small native example without downloading a corpus. The second command checks the long report's embedded arithmetic and saved evidence. The third recalculates the larger scenarios from frozen inputs; it does not build a billion-row index.

Listing 4: Three existing entry points, run from the repository root.

```
.venv/bin/python examples/small_search.py
python3 reports/search-math/verify_math.py
.venv/bin/python -m experiments.project_costs \
  --inputs results/projected-cases-2026-09-11/results.json \
  --output results/my-scenarios \
  --documents 1000000000 --ram-gb 32 64 1000
```

Use the reproduction README for the exact saved-input invocation and full-data runs. A full rerun needs the dataset, embedding cache, and documented environment. The report’s tables are summaries of saved results, not measurements produced by recompiling the PDF.

References

- [1] Exa Team. *How we built a web-scale vector database*. 2024.
<https://exa.ai/blog/building-web-scale-vector-db>.
- [2] A. Kusupati et al. *Matryoshka Representation Learning*. 2022.
<https://arxiv.org/abs/2205.13147>.
- [3] Microsoft. *MS MARCO*. Passage collection and benchmark resources.
<https://microsoft.github.io/msmarco/>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model documentation.
<https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.
- [5] Local experimental record, 10–11 September 2026. Source checkpoint `f16ca66`; `reports/search-math/evidence/` and the result files it identifies. The concise edition’s README records the selected sources and reproduction scope.